

机器学习基础（一）

目 录

1 引言：为什么 HPC 要学机器学习	3
2 从线性回归到神经网络	3
2.1 线性回归：最简单的模型	3
2.2 引入非线性：激活函数	4
2.3 从单层到多层	4
3 训练神经网络	4
3.1 梯度下降	4
3.2 优化器演进	4
3.2.1 SGD + Momentum	4
3.2.2 AdaGrad	4
3.2.3 Adam	5
3.3 反向传播	5
3.4 过拟合与欠拟合	6
4 前馈神经网络	6
4.1 线性层	6
4.2 激活函数详解	6
4.2.1 Sigmoid	6
4.2.2 ReLU	6
4.2.3 Softmax	6
5 卷积神经网络	7
5.1 计算机视觉任务	7
5.2 卷积运算	7
5.3 Padding 与 Stride	7
5.4 多通道卷积	7
5.5 池化	7
5.6 经典 CNN 演进	8
5.7 残差连接	8
6 循环神经网络	8
6.1 序列数据	8
6.2 RNN 结构	8
6.3 BPTT	8
7 Attention 与 Transformer	9
7.1 Scaled Dot-Product Attention	9
7.2 Multi-Head Attention	9

7.3 Self-Attention	9
7.4 Transformer	9
7.5 Llama 与 Mamba	9
8 正则化与归一化	10
8.1 正则化	10
8.2 Dropout	10
8.3 Batch Normalization	10
9 实战：LeNet 训练	10
10 本章你将学会	12
11 要点速查	12
12 小结	12

1 引言：为什么 HPC 要学机器学习

深度学习已经渗透到我们日常使用的各种产品中：GitHub Copilot 帮你写代码，ChatGPT 和 DeepSeek 回答你的问题，DLSS 实现游戏超分辨率，Stable Diffusion 生成艺术图片，Sora 生成视频。这些应用的背后都依赖大规模计算资源，而 HPC 正是支撑这一切的底座。

传统方法 vs 机器学习：

1. 传统方法：手工设计规则和特征，可解释，容量低，数据测试不充分
2. 机器学习：数据驱动，自动学习规则与特征，容量高，可解释性差

传统方法关注“如何手工设计规则”，而机器学习关注“如何将问题形式化，然后自动推导解决方案”。

直觉 为什么 HPC 课程要讲 ML？ML（尤其是 DL）是一类需要大量计算力的“严肃”应用，它和其它领域专用应用一样，既可以使用传统优化技术来加速训练，也可以反过来用 ML 指导系统优化。由此催生了 **MLSys**（机器学习系统）这一新兴研究方向，包括分布式学习（Distributed Learning）、推理加速（Inference Acceleration）、领域专用语言与架构（DSL & Architecture）、自动化系统（AutoSys）等。

本章聚焦：ML/DL 基础（做什么），包括模型、损失与优化器，CNN、Attention、LLM、框架（如 PyTorch）以及高效 AI 技术。不涉及复杂网络的现代 DL 技术、理论分析（为什么这样做？为什么有效？）和应用部署（如何部署 ChatGPT/Stable Diffusion）。

2 从线性回归到神经网络

2.1 线性回归：最简单的模型

我们从最简单的模型开始。给定输入 x 和权重 w ，**线性回归**（Linear Regression）的预测值为：

$$\hat{y} = w^T x + b \quad (2.1)$$

为了衡量预测的好坏，使用 **MSE**（Mean Squared Error，均方误差）作为损失函数：

$$L = \left(\frac{1}{N}\right) \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (2.2)$$

传统优化通过令 $L' = 0$ 解析求解。但当参数量增大时，解析方法不可行，需要 **梯度下降**（Gradient Descent）。

直觉 线性回归虽然简单，但它包含了深度学习的全部核心要素：模型（预测公式）、损失函数（MSE）和优化方法（求导令零或梯度下降）。理解了这个最简模型，后面的复杂网络只是在每个环节上做扩展。

2.2 引入非线性：激活函数

直觉 如果只是把线性层堆叠起来，无论堆多少层，结果仍然等价于一个线性变换。要让模型有能力拟合复杂的非线性关系，必须在层间加入 **激活函数** (Activation Function)。

没有激活函数时，两层线性层叠加：

$$o = W_2(W_1x + b_1) + b_2 = W_2W_1x + b' \quad (2.3)$$

仍等价于单层线性变换。加入激活函数 σ 后：

$$h = \sigma(W_1x + b_1) \quad (2.4)$$

模型才具备了非线性表达能力。

2.3 从单层到多层

将多个带激活函数的线性层堆叠起来，就构成了 **MLP** (Multi-Layer Perceptron, 多层感知机)。参数量随之大幅增长，如何高效优化这些参数？答案是梯度下降。

3 训练神经网络

3.1 梯度下降

梯度下降 (Gradient Descent) 的更新规则：沿损失函数梯度的反方向更新参数。

$$w, b \leftarrow w, b - \eta \nabla L(w, b) \quad (3.1)$$

其中 η 是 **学习率** (Learning Rate)，控制每一步的步长。学习率是一个 **超参数** (Hyperparameter)：用于控制学习过程但不通过训练学习的参数。

直觉 学习率的选择是关键：太小时收敛极慢，太大时震荡甚至发散。就像下山，步子太小走不快，步子太大可能跳过谷底跳到对面的山坡上去。

常用的学习率调度策略有 **Linear Warmup** (线性预热，开始用小学习率逐步增大) 和 **Linear Decay** (线性衰减，逐步减小学习率)。

3.2 优化器演进

3.2.1 SGD + Momentum

标准 **SGD** (Stochastic Gradient Descent, 随机梯度下降)：

$$W \leftarrow W - \eta \nabla L(W) \quad (3.2)$$

SGD + Momentum (动量法) 引入“速度” v 作为梯度的滑动平均：

$$v \leftarrow \rho v + \nabla L(W), \quad W \leftarrow W - \eta v \quad (3.3)$$

直觉 ρ 为摩擦系数 (典型值 0.9 或 0.99)。动量帮助加速收敛、跳出局部极小值，就像小球在斜面上滚下时积累了惯性，即使遇到小坑也能冲过去。

3.2.2 AdaGrad

AdaGrad 累积梯度平方：

$$s \leftarrow s + (\nabla L(W))^2, \quad W \leftarrow W - \eta \frac{\nabla L(W)}{\sqrt{s} + \varepsilon} \quad (3.4)$$

直觉 其本质是对每个参数维度自适应调节学习率：对“陡峭”方向阻尼（步长小），对“平坦”方向加速（步长大）。但步长会随时间单调衰减到零，最终无法继续更新。

3.2.3 Adam

Adam 结合了动量与自适应学习率，是当前最常用的优化器：

$$v \leftarrow \beta_1 v + (1 - \beta_1) \nabla L(W) \quad (3.5)$$

$$s \leftarrow \beta_2 s + (1 - \beta_2) (\nabla L(W))^2 \quad (3.6)$$

做偏差修正（因为初始时 v 和 s 偏向于零）：

$$v' = \frac{v}{1 - \beta_1^i}, \quad s' = \frac{s}{1 - \beta_2^i} \quad (3.7)$$

最终更新：

$$W \leftarrow W - \eta \frac{v'}{\sqrt{s'} + \varepsilon} \quad (3.8)$$

例 Adam 的推荐起点： $\beta_1 = 0.9, \beta_2 = 0.999, \eta = 10^{-3}$ 或 5×10^{-4} 。这是许多模型的好起点！

3.3 反向传播

手动计算梯度代价高且不灵活。**反向传播**（Backpropagation）利用**链式法则**（Chain Rule）自动计算梯度：

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \times \frac{\partial y}{\partial x} \quad (3.9)$$

例 取 $f = (x + y)z$ ，令 $g = x + y$ ，则 $f = gz$ 。取 $x = -2, y = 5, z = -4$ 。

前向计算：

- $g = -2 + 5 = 3$
- $f = 3 \times (-4) = -12$

反向求梯度：

- $\partial_{\frac{f}{\partial}} f = 1$
- $\partial_{\frac{f}{\partial}} g = z = -4$
- $\partial_{\frac{f}{\partial}} z = g = 3$
- $\partial_{\frac{f}{\partial}} x = \left(\partial_{\frac{f}{\partial}} g \right) \times \left(\partial_{\frac{g}{\partial}} x \right) = -4 \times 1 = -4$
- $\partial_{\frac{f}{\partial}} y = \left(\partial_{\frac{f}{\partial}} g \right) \times \left(\partial_{\frac{g}{\partial}} y \right) = -4 \times 1 = -4$

可以看到，反向传播从输出端开始，逐节点计算局部梯度，再用链式法则向前传播。这就是所有深度学习框架自动微分的核心原理。

主流深度学习框架如 *PyTorch*、*TensorFlow*、*JAX*、*MindSpore* 都内置了自动微分引擎，开发者无需手动推导梯度。

3.4 过拟合与欠拟合

模型容量过低导致 **欠拟合** (Underfitting)，模型无法拟合训练数据；容量过高则 **过拟合** (Overfitting)，模型死记训练数据而非学到泛化规律。我们需要在两者之间找到最优容量点。

为此，将数据集划分为三部分：

1. **训练集** (Training Set)：用于学习模型参数
2. **验证集** (Validation Set)：用于选择超参数和模型选择
3. **测试集** (Test Set)：用于最终评估模型性能

4 前馈神经网络

4.1 线性层

线性层 (Linear Layer)，又称线性投影或全连接层 (Fully-Connected Layer)：

$$y = xW + b \quad (4.1)$$

配置项包括输入通道数 (Input Channel) 和输出通道数 (Output Channel)，参数为权重 W 和偏置 b 。FFN (Feedforward Network, 前馈网络) 由多个线性层堆叠构成。

4.2 激活函数详解

4.2.1 Sigmoid

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}} \quad (4.2)$$

将输出压缩到 $[0, 1]$ 区间。其导数为：

$$\left(\frac{d}{dx}\right) \text{sigmoid}(x) = \text{sigmoid}(x)(1 - \text{sigmoid}(x)) \quad (4.3)$$

Sigmoid 的导数最大值仅为 0.25，深层网络中梯度连乘后容易消失。

4.2.2 ReLU

$$\text{ReLU}(x) = \max(x, 0) \quad (4.4)$$

简单高效，计算量小，在正半轴梯度恒为 1，有效缓解梯度消失问题，是现代深度学习最常用的激活函数。

4.2.3 Softmax

$$\text{softmax}(o)_i = \frac{e^{o_i}}{\sum_{j=1}^k e^{o_j}} \quad (4.5)$$

将 k 维 **logits** 转换为概率分布，所有分量之和为 1，常用于多分类任务的输出层。

5 卷积神经网络

5.1 计算机视觉任务

CNN (Convolutional Neural Network, 卷积神经网络) 广泛应用于图像分类、目标检测、语义分割等 CV (Computer Vision, 计算机视觉) 任务。其核心思想是通过卷积操作提取空间特征。

5.2 卷积运算

二维离散卷积:

$$y_{i,j} = \sum_{m=0}^k \sum_{n=0}^k w_{m,n} x_{i-\frac{k}{2}+m, j-\frac{k}{2}+n} \quad (5.1)$$

直觉 卷积核在输入上滑动，每个位置做逐元素相乘再求和，得到一个输出值。核的权重是可学习参数，通过训练自动提取有用的特征模式（如边缘、纹理等）。

例 取 3×3 输入和 2×2 卷积核（无 padding, stride = 1）:

输入: $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$, 核: $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$

计算左上角输出: $y_{0,0} = 1 \times 1 + 2 \times 0 + 4 \times 0 + 5 \times 1 = 6$

完整输出: $\begin{pmatrix} 6 & 8 \\ 12 & 14 \end{pmatrix}$

可以看到输出尺寸从 3×3 缩小到 2×2 , 因为 $\frac{3-2}{1} + 1 = 2$ 。

关键配置项:

1. **Kernel Size** (卷积核大小): $k_h \times k_w$
2. **Channel** (通道数): 输入通道 c_i , 输出通道 c_o
3. **Stride** (步长): 卷积核每次移动的距离
4. **Padding** (填充): 在输入边缘补零以控制输出尺寸

5.3 Padding 与 Stride

Padding (填充) 避免丢失边缘像素，常用零填充 (Padding = 0) 或最近邻填充 (Padding = 1)。

Stride (步长) 控制卷积核移动的步长。stride 越大，输出尺寸越小。例如 stride $x, y = 2, 3$ 时，水平和垂直方向的步长分别为 2 和 3。

5.4 多通道卷积

输入数据可能包含多个通道 c_i (如 RGB 图像有 3 个通道)。此时卷积核形状为 $c_i \times k_h \times k_w$ 。

当需要输出 c_o 个通道时，使用 c_o 个卷积核，总核形状为 $c_o \times c_i \times k_h \times k_w$ 。

5.5 池化

池化 (Pooling) 使用固定形状窗口在输入上滑动，无可学习参数:

1. **Max Pooling** (最大池化): 取窗口内最大值
2. **Avg Pooling** (平均池化): 取窗口内均值

5.6 经典 CNN 演进

网络	年份	关键创新
LeNet	1998	早期 CNN 雏形
AlexNet	2012	引爆深度学习浪潮
VGGNet	2014	小滤波器 3×3 , 更深 16-19 层
GoogLeNet	2014	Inception Block: $1 \times 1 + 3 \times 3 + 5 \times 5$ 多尺度
ResNet	2016	残差连接, 解决梯度消失, 超深网络

GoogLeNet 的 Inception Block 使用 1×1 、 3×3 、 5×5 三种卷积核并行提取不同空间尺度的信息, 并在通道维度拼接。其中 1×1 卷积还用于减少通道数, 降低计算量。

5.7 残差连接

ResNet (He et al., 2016) 的核心创新: 学习残差映射 $f(x) - x$ 而非直接映射 $f(x)$ 。

直觉 当网络已经很深时, 梯度在反向传播中会逐渐消失。残差连接让梯度可以“跳过”中间层直达前方, 使得超深网络 (如 100+ 层) 成为可能。形式上, 保证 $F_i \subseteq F_{i+1}$ (令残差映射为零时, 深层等价于浅层), 因此加深网络至少不会变差。

6 循环神经网络

6.1 序列数据

序列数据包括文本、音频、股票报价、视频帧、手势动作等, 甚至图像也可视为像素序列。这类数据的关键特征是: 当前时刻的输出依赖于之前时刻的信息。

6.2 RNN 结构

RNN (Recurrent Neural Network, 循环神经网络) 通过隐藏状态 h_t 记忆序列信息:

$$h_t = f(h_{t-1}, x_t; w_h), \quad o_t = g(h_t; w_o) \quad (6.1)$$

每一步更新隐藏状态并产生输出, 权重 w_h 和 w_o 在所有时间步共享。将 RNN 在时间步上展开, 就得到了展开计算图。

6.3 BPTT

Backpropagation Through Time (BPTT, 沿时间反向传播) 将 RNN 在时间步上展开后进行反向传播。总损失为各时间步损失之和:

$$L = \left(\frac{1}{T}\right) \sum_{t=1}^T l(y_t, o_t) \quad (6.2)$$

直觉 梯度按链式法则沿时间维度传播。但当序列很长时, 梯度需要连乘很多次, 容易消失或爆炸。这就是 RNN 难以处理长序列的根本原因。

RNN 的梯度消失问题后来由 LSTM 和 GRU 等变体缓解，最终被 Transformer 取代。

7 Attention 与 Transformer

7.1 Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (7.1)$$

直觉 Attention 的核心思想是“查询-检索”： Q (Query, 查询) 和 K (Key, 键) 计算相似度，用相似度对 V (Value, 值) 加权求和。缩放因子 $\sqrt{d_k}$ 用于稳定梯度：当 d_k 较大时，点积 QK^T 的值变大，softmax 输出趋近于 one-hot，梯度趋近于零。

7.2 Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O \quad (7.2)$$

其中 $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ 。

直觉 每个 head 使用不同的线性投影，让模型从不同子空间关注不同方面的信息。例如在 NLP 中，有的 head 关注语法关系，有的关注语义相似性。

7.3 Self-Attention

当 $Q = K = V = X$ 时，即 **Self-Attention** (自注意力)，查询、键、值都来自同一输入，捕捉序列内部的关系。

关键易错点：Self-Attention 对输入顺序不敏感（置换等变），必须加 **位置编码** (Position Encoding)，否则无法区分序列顺序。常用方法有 $\sin\text{-}\cos$ 编码和学习式编码。

7.4 Transformer

Transformer (Vaswani et al., 2017) 采用 Encoder-Decoder 结构：

1. **Encoder**: 使用 Self-Attention 编码输入序列
2. **Decoder**: 使用 **Masked Multi-Head Attention** 防止训练时看到未来信息

直觉 Decoder 的 Masked MHA 是 LLM 自回归训练的关键：生成第 t 个 token 时只能看到前 $t-1$ 个 token，不能“偷看”未来信息。这通过将注意力矩阵的上三角部分置为 $-\infty$ 来实现。

7.5 Llama 与 Mamba

当代主流 LLM 架构 **Llama** (Hugo et al., 2023) 采用 Decoder-only 结构：

1. **RoPE** (Rotary Position Embedding, 旋转位置编码) 编码 Q 和 K
2. **RMSNorm** 替代 LayerNorm，计算更高效

3. **SwiGLU** 替代 ReLU，提升表达能力

Mamba (Albert et al., 2023) 则基于 **SSM** (State Space Model, 状态空间模型)，以 $x(t) \rightarrow y(t)$ 的方式建模，追求更高效的模型架构。

8 正则化与归一化

8.1 正则化

在损失函数中加入正则项以防止过拟合：

$$L'(W) = L(W) + \lambda R(W) \quad (8.1)$$

1. **L1 正则化**: $R(W) = \sum_k \sum_l |W_{k,l}|$

2. **L2 正则化**: $R(W) = \sum_k \sum_l W_{k,l}^2$

直觉 L1 倾向于产生稀疏权重（部分参数精确归零），适合特征选择；L2 倾向于产生小而均匀的权重，防止某个参数过大。

8.2 Dropout

Dropout (Srivastava et al., 2014) 在每次前向传播中随机将部分神经元置零，丢弃概率是超参数。这迫使网络不过度依赖任何单个神经元，起到正则化效果。

8.3 Batch Normalization

Batch Normalization 的动机：输入 x 的分布可能不好（不居中于零、各维度尺度不同）。对每个特征维度进行归一化。

输入 $x \in \mathbb{R}^{N \times D}$ ，对第 j 维：

$$\mu_j = \left(\frac{1}{N}\right) \sum_{i=1}^N x_{i,j} \quad (8.2)$$

$$\sigma_j^2 = \left(\frac{1}{N}\right) \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad (8.3)$$

归一化输出：

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \quad (8.4)$$

归一化方法有多种变体 (*LayerNorm*、*GroupNorm* 等)，在 *Transformer* 中通常使用 *LayerNorm* 而非 *BatchNorm*，因为 *BatchNorm* 依赖 *batch* 内统计，不适合变长序列。

9 实战：LeNet 训练

下面用 PyTorch 实现 LeNet 的训练流程。LeNet 是最早的 CNN 之一 (LeCun et al., 1998)，结构简洁，适合入门。

```
import torch
import torch.nn as nn
```

```

class LeNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, 5)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = torch.max_pool2d(x, 2)
        x = torch.relu(self.conv2(x))
        x = torch.max_pool2d(x, 2)
        x = x.view(x.size(0), -1)
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = self.fc3(x)
        return x

model = LeNet()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(10):
    for images, labels in train_loader:
        outputs = model(images)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

```

逐行批注：

1. `nn.Conv2d(1, 6, 5)`: 输入 1 通道, 输出 6 通道, 5×5 卷积核
2. `nn.Linear(16 * 5 * 5, 120)`: 将卷积输出展平后映射到 120 维, $16 \times 5 \times 5$ 是展平后的特征数
3. `torch.relu(...)`: 对卷积输出应用 ReLU 激活函数
4. `torch.max_pool2d(x, 2)`: 2×2 最大池化, 尺寸减半
5. `x.view(x.size(0), -1)`: 将多维特征展平为二维, `-1` 自动推断
6. `nn.CrossEntropyLoss()`: 交叉熵损失, 内部先做 softmax 再取负对数似然
7. `torch.optim.Adam(..., lr=1e-3)`: Adam 优化器, 学习率 10^{-3} , 使用默认 $\beta_1 = 0.9, \beta_2 = 0.999$
8. `optimizer.zero_grad()`: 清空上一步的梯度缓存
9. `loss.backward()`: 反向传播, 自动计算所有参数梯度
10. `optimizer.step()`: 按 Adam 更新规则更新参数

例 取一个 mini-batch ($N = 4$, 10 类分类), 假设模型对第一个样本的 logits 为 $o = (0.1, 0.2, 0.7, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0)$, 标签为 2。

先做 softmax: $p_2 = \frac{e^{0.7}}{e^{0.1}+e^{0.2}+e^{0.7}+7 \times e^0} = \frac{2.014}{1.105+1.221+2.014+7} \approx 0.180$

交叉熵损失: $l = -\log(p_2) \approx -\log(0.180) \approx 1.715$

反向传播会计算 $\partial_{\theta}^l o_2$ 等梯度, 然后 Adam 按动量和自适应学习率更新参数。

10 本章你将学会

1. 理解深度学习的三件套：**模型定义、损失函数、优化器**，能说出梯度下降和 Adam 的更新公式
2. 能够推导反向传播的链式法则，并用小数值例子手动计算梯度
3. 掌握 CNN 的卷积、池化、多通道运算，理解 Padding 和 Stride 对输出尺寸的影响
4. 理解 Attention 机制和 Transformer 结构，能解释 Self-Attention 为什么需要位置编码
5. 了解正则化（L1/L2、Dropout、BatchNorm）在防止过拟合中的作用

11 要点速查

要点	说明
梯度下降	$W \leftarrow W - \eta \nabla L(W)$
Adam 推荐	$\beta_1 = 0.9, \beta_2 = 0.999, \eta = 10^{-3}$
链式法则	$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \times \frac{\partial y}{\partial x}$
Attention	$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
无激活=线性	两层线性叠加仍等价单层，必须加激活
Self-Attention	必须加位置编码，否则不区分顺序
Masked MHA	Decoder 训练时 mask 未来位置防信息泄露
AdaGrad 缺陷	步长单调递减到零
残差连接	学习 $f(x) - x$ 缓解梯度消失
L1/L2	$\sum W / \sum W^2$
BatchNorm	按特征维度归一化到均值 0、方差 1

12 小结

本章覆盖了深度学习的基础知识：从线性回归到梯度下降与优化器（SGD/Momentum/AdaGrad/Adam），从激活函数到 FFN/CNN/RNN/Transformer 的模型架构，再到正则化与归一化技术。核心三件套，**模型定义、损失函数、优化器**，是所有深度学习的基础。理解了这些，你才能进入下一章的系统优化世界。

讲义基于 HPC101-2025 Day10「机器学习基础 1」课程内容编写